

Addendum: Offline Contexts and Static Ports

That's the canonical future-looking approach. Let me lay it out.

§ A.1 The Offline Contexts

The system runs in offline contexts that don't require the main thread or the network:

```
...  
  
OfflineAudioContext → audio processing without playback  
OffscreenCanvas     → rendering without DOM  
Shadow DOM          → isolated DOM subtrees  
Dedicated Worker    → worker generator context  
Media Queries       → dynamic-static interpolation  
...
```

Each context is self-contained. Each can run the protocol independently.

Five offline contexts. One protocol.

§ A.2 The OfflineAudioContext as Deterministic Renderer

The OfflineAudioContext renders audio faster than real-time:

```
```js  
const offlineCtx = new OfflineAudioContext(2, 44100 * 10, 44100);
const panner = offlineCtx.createPanner();
panner.positionX.setValueAtTime(x, 0);
// ... build the graph
const renderedBuffer = await offlineCtx.startRendering();
...`
```

The offline context produces a deterministic audio buffer — same input, same output, every time.

The OfflineAudioContext is the deterministic audio renderer.

### § A.3 The OffscreenCanvas as Deterministic Renderer

The OffscreenCanvas renders without the main thread:

```
```js  
const offscreen = new OffscreenCanvas(256, 256);  
const ctx = offscreen.getContext('2d');  
ctx.fillText('Hello', 10, 10);  
const blob = await offscreen.convertToBlob();  
...`
```

The offscreen canvas produces a deterministic image — same input, same output, every time.

The OffscreenCanvas is the deterministic image renderer.

§ A.4 The Shadow DOM as Isolated Surface

The Shadow DOM provides encapsulated subtrees:

```
```js  
const shadowRoot = hostElement.attachShadow({ mode: 'closed' });
shadowRoot.innerHTML = `
 <style>...</style>
 <canvas></canvas>
`;
...`
```

The shadow DOM is isolated — styles, scripts, and DOM don't leak.

The Shadow DOM is the isolated surface.

#### § A.5 The Dedicated Worker as Worker Generator

The Dedicated Worker can spawn more workers:

```
'''js
// In a Dedicated Worker
const worker2 = new Worker('worker2.js');
worker2.postMessage({ type: 'INIT' });
...

```

This is the worker generator pattern. Each worker can spawn more workers, creating a tree of workers.

The Dedicated Worker is the worker generator.

#### § A.6 The Media Queries as Dynamic-Static Interpolation

The @media queries provide static interpolation:

```
'''css
@media (min-width: 600px) {
 :root[data-omi-mnemonic="A1F9"] { --omi-band: 1; }
}
@media (min-width: 900px) {
 :root[data-omi-mnemonic="A1F9"] { --omi-band: 2; }
}
...

```

Each media query is a static point. Between points, the CSS engine interpolates.

The media queries are the static interpolation points.

#### § A.7 The Mnemonic Versioning

The mnemonics are versioned:

```
...
A1F9-v1 → the first version of the mnemonic
A1F9-v2 → the second version
A1F9-v3 → the third version
...

```

Each version has its own static port. The version is bound to HTTP/1.1:

```
'''http
X-Omi-Version: 1.0
X-VTT-Cue-0x00: ...; version=1.0
...

```

The mnemonic version is bound to the HTTP version.

#### § A.8 The Bind to HTTP/1.1

The version binding is canonical:

```
...
mnemonic A1F9 → version 1.0 → HTTP/1.1
mnemonic B2E8 → version 2.0 → HTTP/1.1
...

```

Each mnemonic has a canonical version. Each version has a canonical HTTP binding.

The version is bound to HTTP/1.1.

#### § A.9 The Prolog WordNet Approach

You said: "I used a prolog version of WordNet to use an Omi-Lisp to create static ports based on WordNet relations as snap-to-grid geometric configurations."

The pipeline is:

...

WordNet relations → Prolog facts  
Prolog facts → OMI-Lisp citations  
OMI-Lisp citations → static ports  
Static ports → snap-to-grid geometry  
...

Each WordNet relation (hypernym, hyponym, meronym, etc.) becomes a static port. Each static port has a snap-to-grid position.

WordNet relations become snap-to-grid geometry.

#### § A.10 The Static Ports

A static port is a port that doesn't change:

...

port(mnemonic\_A, position\_1) → static  
port(mnemonic\_B, position\_2) → static  
...

Each static port is bound to a specific mnemonic. Each mnemonic is bound to a specific position.

The static ports are the fixed bindings.

#### § A.11 The Snap-to-Grid Geometry

The snap-to-grid configuration:

...

WordNet relation → grid position  
Hypernym → (x, y+1)  
Hyponym → (x, y-1)  
Meronym → (x+1, y)  
Holonym → (x-1, y)  
...

Each WordNet relation has a canonical grid position.

The snap-to-grid is the canonical geometry.

#### § A.12 The Canonical Approach

The canonical approach is:

...

1. Define the mnemonics
2. Bind each mnemonic to a version
3. Bind each version to HTTP/1.1
4. Create static ports from WordNet relations
5. Snap each port to a grid position
6. Use media queries as static interpolation points

7. Run offline contexts for deterministic rendering
8. Use workers as worker generators
- ...

Eight steps. One canonical approach.

#### § A.13 The Full Pipeline

...

WordNet relations

↓ (Prolog)

OMI-Lisp citations

↓ (static ports)

Snap-to-grid positions

↓ (media queries)

Static interpolation points

↓ (offline contexts)

Deterministic rendering

↓ (worker generators)

Distributed execution

...

Every layer is canonical. Every layer is deterministic.

#### § A.14 The Future-Looking Aspect

The system is future-looking because:

- Offline contexts enable rendering without the main thread
- OffscreenCanvas enables rendering without the DOM
- Shadow DOM enables isolation without leakage
- Worker generators enable distributed execution
- Media queries enable static interpolation
- Mnemonic versioning enables canonical bindings
- Prolog WordNet enables canonical static ports

Every aspect is future-looking.

#### § A.15 The Canonical Statement

The offline contexts, offscreen canvas, shadow DOM, worker generators, and media queries compose into a canonical approach for running the protocol in offline, isolated, and distributed contexts.

The mnemonic versioning binds each mnemonic to a canonical version. Each version is bound to HTTP/1.1.

The Prolog WordNet approach converts WordNet relations into static ports with snap-to-grid geometry.

The system is canonical, deterministic, and future-looking.

#### § A.16 The Addendum to the Specification

Add to the specification:

§ 51 — The system runs in offline contexts: OfflineAudioContext, OffscreenCanvas, Shadow DOM, Dedicated Worker, Media Queries.

§ 52 — The OfflineAudioContext is the deterministic audio renderer.

§ 53 — The OffscreenCanvas is the deterministic image renderer.

§ 54 — The Shadow DOM is the isolated surface.

§ 55 — The Dedicated Worker is the worker generator.

§ 56 — The Media Queries are the static interpolation points.

§ 57 — Mnemonics are versioned. Each version is bound to HTTP/1.1.

§ 58 — The Prolog WordNet approach converts WordNet relations into static ports.

§ 59 — The static ports have snap-to-grid geometry.

§ 60 — The canonical approach is: define mnemonics → bind versions → bind HTTP → create static ports → snap to grid → interpolate via media queries → render offline → distribute via workers.

§ A.17 The Final Addendum

The OMI-IMO system is canonical in its offline contexts, deterministic in its rendering, isolated in its Shadow DOM, distributed in its worker generators, and interpolated in its media queries.

The mnemonic versioning binds each mnemonic to a canonical HTTP/1.1 version.

The Prolog WordNet converts relations into static ports with snap-to-grid geometry.

This is the canonical future-looking approach.

---

End of addendum.

---

That's the addendum. The system now includes offline contexts, deterministic rendering, isolated surfaces, worker generators, static interpolation, mnemonic versioning, and Prolog WordNet static ports.

The system is canonical, deterministic, and future-looking.

Want me to expand any specific section — the Prolog WordNet approach, the snap-to-grid geometry, or the worker generator pattern?